

Manuel de configuration - INJS-LMD 2026

Parcours demonstrable : candidature -> admission -> inscription -> evaluations -> jury -> diplomation

Fichier : ManuelConfiguration-INJS-LMD2026.txt (racine du depot)

Inspection de reference : 2026-10-05 (branche main, commit af80a869)

Donnees : fictives, prefixe DEMO. Aucun secret reel.

0. Resume executif

But du manuel :

Permettre a une personne technique qui decouvre le projet de configurer, lancer et verifier, de bout en bout, le cycle academique LMD d'une etudiante fictive « Awa KONE -- DONNEE DE DEMONSTRATION » :
campagne -> candidature -> admission -> inscription administrative et pedagogique -> notes -> jury -> diplomation (documents + registre).

Etat constate du depot (verifie le 2026-10-05, non destructif) :

- Monorepo a plat : backend/ (Django 5.1.4 + DRF 3.15.2, 20 apps), frontend/ (React 18.2 + Vite 7), qr_badge_mobile/ (Flutter), docs/, scripts/, arena/, docker/.
- 250 migrations Django, toutes appliquees ([X], 0 x []) sur la base locale injs_lmd_current (PostgreSQL 16.14 Homebrew, 127.0.0.1:5432).
- Sante confirmee : GET http://127.0.0.1:8000/api/health/ -> {"status":"ok","database":"ok"} ;
GET http://localhost:3000/ -> 200.
- `python manage.py check` : 1 warning urls.W005 (namespace dashboard_api non unique), 0 erreur.
- Ports figes projet : API Django 8000, front Vite 3000 (strictPort:true), Postgres 5432 (local INJS-LMD) vs 5436 (Docker, autre instance).
- Authentification JWT (SimpleJWT 5.4.0) + 12 roles canoniques (backend/authentication/models.py, classe User.Role).
- Modules metier LMD presents et routés : admissions, scholarite, evaluations (suiviEvaluation + evaluations), jurys, graduation, finances_etudiantes, referentiels, parametres, edts, stages.
- Script E2E de reference (non execute ici) :
backend/tests_e2e_injs_2026_2027.py (parcours complet + 12 cas negatifs).
- seed_data.py historique SYGEP-CPFAE avec mots de passe en clair :
NE PAS utiliser tel quel en demo/prod (voir section 9).

Demarrage rapide (chemins confirmes) :

```
# Terminal 1 -- API (depuis racine du depot)
cd backend && python3 manage.py check
python3 manage.py showmigrations | grep -c "[X]" # attendu : 250
curl -s http://127.0.0.1:8000/api/health/
# Lancement local fige (voir section 4 pour le choix du lanceur) :
# API : http://127.0.0.1:8000 Front : http://localhost:3000
```

Prerequis critiques (detail section 3) :

1. **Python 3.12 + PostgreSQL 16 accessible (ou USE_SQLITE=1 en bac a sable).**
2. **Node 20/22 + npm (front Vite, port 3000 libre).**
3. **Fichier backend/.env renseigne SANS secret reel (placeholders).**
4. **Base injs_lmd_current + role injs_user (ne jamais DROP/TRUNCATE/FLUSH).**
5. **Sauvegarde avant toute operation massive : bash scripts/backup_injs_lmd.sh**

1. Perimetre et hypotheses verifiees

1.1 Version/date d'inspection

- Date : 2026-10-05. Branche : main. HEAD : af80a869 (manuel utilisateurs).
- Methode : lectures + commandes NON destructives uniquement (ls, grep, manage.py check/showmigrations, curl health, psql SELECT, pg_isready).
- Aucune migration, aucun seed, aucune ecriture en base.

1.2 Technologies detectees et preuves (fichiers observes)

- Backend : Django 5.1.4 + DRF 3.15.2 + SimpleJWT 5.4.0 + drf-spectacular 0.28.0. Preuves : backend/requirements.txt (31 lignes), backend/README.md, backend/config/settings.py l.116-155 INSTALLED_APPS.
- Base : PostgreSQL (ENGINE django.db.backends.postgresql, settings ~l.191). Local : PostgreSQL 16.14 Homebrew, base injs_lmd_current, user injs_user, 250 migrations appliquees. Compose : postgres:16-alpine. Doc historique : image 15 dans backend/compose.yml (ecart sans impact).
- Cache/services : Redis 7 + RabbitMQ 3-management (docker-compose.yml) ; repli local cache fichier/memoire (docs/ARCHITECTURE.md). Non requis.
- Frontend : React 18.2 + Vite 7 + react-router-dom 6.22 (frontend/package.json), config frontend/vite.config.js (port 3000, strictPort true, proxy /api -> 127.0.0.1:8000).
- Mobile : qr_badge_mobile/ (Flutter, hors perimetre demo web).
- Exports : reportlab 4.2.5 (PDF), openpyxl 3.1.5, python-docx 1.1.2, Pillow 11.1.0, qrcode 8.0.
- API docs : /api/v1/schema/ + /api/v1/docs/ (backend/config/urls.py), alias legacy /api/schema/. Sante : GET /api/health/ (backend/config/health.py -> status, database, timestamp).
- Auth : JWT Bearer (docs/API_ENDPOINTS.md). Routes reelles backend/authentication/urls.py : login/, mfa/*, token/refresh/, logout/, me/, roles/, capabilities/, me/change-password/, users/.
- Env : .env.example (racine), backend/env.exemple, backend/.env local existant (tunnel 29/09/2026, DEBUG=False).

1.3 Fonctions disponibles (modeles + urls + pages verifiees)

- Referentiels LMD (scolarite) : AnneeAcademique, TypeFormation, Niveau, Parcours, Semestre, RegimeEtudes, StatutEtudiant, Groupe, Maquette, UE, ECUE, RegleValidationLMD ; formations (RefFormation...) ; referentiels (RefSocle) ; parametres (Parametre).
URLs : /api/scolarite/ref/<ressource>/, /maquettes/*, /ues/*, /ecues/*.
Pages : ScolariteDashboard, Maquettes, MaquetteDetail, Referentiels.
- Candidatures/admissions : Candidat, Candidature (12 statuts BROUILLON... ADMIS/REFUSE/ANNULE), PieceCandidature, Admission (ADMIS, ADMIS_SOUS_RESERVE, EN_ATTENTE, REFUSEE, ANNULEE), CampagneAdmission (BROUILLON...ARCHIVEE), Epreuve/Convocation/NoteConcours/ClassementConcours.
URLs : /api/admissions/candidats/, /candidatures/, /candidatures/<pk>/transition/, /pieces/<id>/{deposer,verifier,fichier}, /admissions/ + /decision/ + /annuler/, /campagnes/ + /transition/ + /epreuves/ + /classement/ + /publier/.

Pages : Candidatures, Campagnes, CampagneDetail, Admissions.

- Inscriptions : DossierEtudiant + InscriptionAdministrative (8 statuts, 6 types, unicite VALIDEE, refus annee cloturee) + conversion depuis admission (inscrire_depuis_admission) + pedagogie (generer/ajouter) + groupes (affectations historisees, repartition auto, reinscriptions).

URLs : /api/scolarite/etudiants/, /inscriptions/,
/inscriptions/depuis-admission/, /<pk>/transition/, /radier/,
/changer-parcours/, /<pk>/pedagogie/*, /groupes/*, /reinscriptions/.

Pages : Inscriptions, FicheEtudiant, Groupes, MonEspace.

- Evaluations/notes : app evaluations (sessions, evaluations, composants, epreuves, notes, ECUEResult/UEResult/SemesterResult, ReleveNotes SHA-256, passage niveau) + app suiviEvaluation (notes par module legacy).

URLs : /api/evaluations-academiques/* et /api/evaluations/*.

Pages : EvaluationsModule, NotesModule.

- Jurys : SessionJury (9 statuts PREPARATION->...->PUBLIE, types NORMALE/RATTRAPAGE/RECTIFICATION, maquette verrouillee exiguee) + MembreJury + PropositionJury + DecisionJury + PVJury + NotificationJury.

URLs : /api/jurys/sessions/ + /<pk>/[action,membres,decisions,pv].

Page : Jurys.jsx. Double historique /api/juries/ + /api/jurys/ conserve.

- Diplomatation : Diplome (DRAFT/VALIDATED/REVOKED/REISSUED, PDF gele, conditions = inscription VALIDEE + decision jury session VERROUILLEE/PUBLIEE) + Reedition (append-only) + RegistreDiplomes + ModeleDocument + verification publique par token.

URLs : /api/graduation/diplomes/ + /creer/ + /valider/ + /revoquer/ +
/reedition/ + /pdf/, /registres/, /modeles/, /verifier/<token>/.

Page : Graduation.jsx.

- Finances etudiantes : Tarification, Echeancier(+lignes), Facture, Paiement, Quittance, Remboursement, Relance, RapprochementComptable.

URLs : /api/finances-etudiantes/echeanciers/, /paiements/ + /confirmer/,
/quittances/, /remboursements/, /relances/, /rapprochements/.

Page : FinancesEtudiantes.jsx.

- EDT/presences/stages/admin : edts (contrat lecture seule /api/scolarite/edt/* + /api/edts/ + alias /api/timetable/), presences (QR), stages, administrations/RH/patrimoine. Utiles en P2, non bloquants.

1.4 Limites constatees (sans blocage du fil demo)

- Notifications unifiees : module `notifications` A CREER (README) ; simuler par statut + capture ecran.
- Comptabilite generale/budget : app `comptabilite` A CONSTRUIRE ; sans impact sur parcours etudiant.
- Ne pas reintroduire les interfaces des anciennes apps (regle projet).
- seed_data.py historique SYGEP-CPFAE, mots de passe en clair : NE PAS utiliser tel quel (voir section 9). Demo = jeu DEMO_* section 5.

1.5 Hypotheses a faire valider par l'INJS

H1 Annee demo : libelle 2026-2027, code 2026 (E2E dit 2026-2027, mission dit 2026). Valider calendrier officiel + sessions.

H2 Filiere fictive : STAPS / Management du Sport (inspire E2E, prefixe DEMO). Ne pas presenter comme referentiel officiel.

H3 Seuils LMD (validation UE/ECUE, compensation, credits) : moteurs

RegleValidationLMD + RegleCalcul presents, valeurs a saisir et valider.

H4 Tarifs/frais 2026 : montants fictifs en monnaie locale, echeances et

exonerations a valider.

H5 Modeles de documents (attestation, releve, diplome) : a valider.

H6 Comptes demo : a creer via API/admin (jamais via seed historique),

mots de passe temporaires a faire changer (must_change_password).

2. Plan priorise (P0/P1/P2/P3)

Priorite	Objectif	Action	Dependances	Preuve	Risque / retour
P0	Python/Node operationnels	Verifier python3.12, node 20/22, npm, psql 16	-	versions affichees	Aucun (lecture seule)
P0	Base locale saine	pg_isready 127.0.0.1:5432 + SELECT sur injs_lmd_current	Postgres démarre	1 ligne version + 250 migrations	Lecture seule ; ne jamais DROP/TRUNCATE/FLUSH
P0	Config sans secret reel	Copier .env.example/backend-env.exemple -> .env avec placeholders <...>	-	backend/.env present, aucun secret commis	git status propre ; .env ignore par Git
P0	Socle Django vert	manage.py check + showmigrations (250 x [X])	.env + base	check 0 erreur + health ok	Lecture seule
P0	Sauvegarde pre-demo	bash scripts/backup_injs_lmd.sh + verifier .dump + .sha256	Postgres + droits	dump non vide, pg_restore --list OK	Espace disque ~/Backups/INJS-LMD/data base/
P0	Acces demo par roles	Creer ADMIN/INJS_ADMIN/SECRETARIAT/FORMATEUR/AUDITEUR via admin/API	base + auth	login JWT 200 + /me/ OK	Desactiver (is_active=False), pas de DELETE
P0	Lancement fige	API 8000 + front 3000 (lanceurs arena ou start_dev.sh)	deps installees	/api/health/ ok + front 200	pkil ciblé ; ne pas changer de port
P1	Referentiel 2026	Annee 2026-2027 + Niveaux + Parcours DEMO + Semestres S1/S2 + Maquette VALIDEE/ACTIVE	roles	maquette verrouillee visible	Archiver/cloner, pas de DELETE
P1	Campagne OUVERTE	CampagneAdmission OUVERTE sur (annee, RefFormation, parcours)	annee+formation	transition OUVERTE OK	SUSPENDUE/CLOTUREE reversibles
P1	Candidature AwaKONE	Candidat fictif + Candidature BROUILLON->SOUMISE + pieces deposees/verifiees	campagne	statut SOUMISE puis PIECES_VALIDEES	ANNULE (contrainte unicite hors ANNULE)
P1	Admission	Admission ADMIS (+ decision candidature ADMIS)	pieces validees	permet_inscription=true	ANNULEE + retour EN_ETUDE
P1	Inscription admin+peda	inscrire_depuis_admission -> VALIDEE + pedagogie generer	admission non expiree	FicheEtudiant + matricule INJS26-XXXX	radier/suspendre (jamais DELETE)
P1	Finances demo	Tarification + echeancier + facture + paiement confirme + quittance	inscription	quittance numerotee	remboursement/annulation tracee
P1	Notes	Sessions/composants + saisie notes CC/Examens + resultats ECUE/UE/Semestre	inscriptions peda	moyennes calculees	historique Evaluation GradeHistory
P1	Jury	Session NORMALE + membres + decisions + transitions jusqu'a PUBLIE + PV	notes + maquette verrouillee	PV telechargeable	session RECTIFICATION si erreur
P1	Diplomation	Diplome creer->valider (conditions OK) + pdf + registre + verifier/<token>/	jury PUBLIE + inscription VALIDEE	registre + verification publique OK	revoquer motivee + reedition
P2	Observabilite	logs backend/logs/injs_lmd.log + health + Swagger	services	200 + logs WARNING en prod	Rotation 5x5Mo deja configuree
P2	Reset demo sur	Suppression ciblee DEMO_* + reimport (jamais flush)	sauvegarde	donnees demo absentes puis recreees	Restaurer le .dump si erreur
P2	Tests verts	manage.py test (backend) + vitest run (front, 91 fichiers)	collectstatic prealable	0 echec	Ne pas commiter .pyc/dist
P2	E2E reference	backend/tests_e2e_injs_2026_2027.py (12 cas negatifs + RBAC)	env dedie	rapport OK	Destructif : DEMANDER confirmation avant
P3	Docker Compose	docker compose up --build (8080/8000/admin)	Docker	200 sur / et /api	docker compose down SANS -v (volume protege)
P3	Tunnel/iframe	arena/settings_sandbox.py + bootstrap.sh (aperçu)	sandbox	front 3000 via proxy	Ne pas exposer /admin/ via 3000 (bloque)

3. Prerequis et preparation de l'environnement

3.1 Logiciels et versions (sources : fichiers du depot)

- Python 3.12 (cible CI/Docker ; constate 3.12.13). Preuve : backend/Dockerfile, CI.
Verification : `python3 --version` [code] Attendu : 3.12.x.
- Django 5.1.4 (constate via import). Verification : `python3 -c "import django; print(django.__version__)"` [code]
- PostgreSQL 16 (constate 16.14 Homebrew). Verification : `psql --version ; pg_isready -h 127.0.0.1 -p 5432` [code]
IMPORTANT : 127.0.0.1:5432 = PostgreSQL INJS-LMD local (injs_lmd_current / injs_user).
127.0.0.1:5436 = PostgreSQL Docker, AUTRE instance. Ne pas confondre.
- Node 20/22 + npm (CI dit 22, image front dit 20 ; constate 26.0.0 analyse — utiliser 20/22 en demo).
Verification : `node --version ; npm --version` [code]
- Redis 7 / RabbitMQ : optionnels en demo locale (repli cache fichier/memoire).
- Docker Engine + plugin Compose : optionnel (P3). Verification : `docker --version ; docker compose version` [code]

3.2 Comptes, ports, acces, reseau, stockage, messagerie, documents

- Ports figes (regle projet) : API Django 8000, front Vite 3000, Postgres 5432, Redis 6379.
Si occupe : liberer le port, NE JAMAIS deplacer le serveur.
Verification : `ss -tlnp | grep -E "\":3000[:8000]:5432[:6379]"` [code] (macOS : `lsof -i :8000 -i :3000` [code]).
- Base : host 127.0.0.1, port 5432, db injs_lmd_current, user injs_user, password <DB_PASSWORD_DEMO>.
Verification : `psql -h 127.0.0.1 -p 5432 -U injs_user -d injs_lmd_current -c "SELECT current_database(), current_user;"` [code]
- Stockage fichiers : backend/media/ (pieces, documents), backend/staticfiles/ (non versionne).
Verification : `ls -ld backend/media backend/staticfiles` [code]
- E-mail : console Django en dev (EMAIL_* vides) ; SMTP Mailcow en prod (voir backend/env.exemple).
Demo : laisser vide -> e-mails dans la console. Ne jamais saisir de mot de passe reel.
- Documents : ReportLab/openpyxl/python-docx (deja en requirements). Verification : `import Python`.
- Mobile/QR : hors demo (PUBLIC_QR_SCAN_ENABLED=False par default ; parcours authentifie /api/scan/secure/).

3.3 Environnements distincts (local / demo-staging / production)

- Local : backend/.env (DEBUG=False dans le .env tunnel du 29/09/2026 ; DEBUG=true possible en dev pur),
ALLOWED_HOSTS=127.0.0.1,localhost, FRONTEND_URL=http://127.0.0.1:3000, CORS vide (proxy /api meme origine).
- Demo/staging : variables equivalentes avec hotes de demo, SECRET_KEY generee (<DJANGO_SECRET_KEY_DEMO>),
mots de passe <DB_PASSWORD_DEMO>, jamais de dump prod.
- Production : domaine https://injs.badge-qr-code.pro (CORS/CSRF stricts, HSTS, cookies Secure, SMTP reel).
Toute commande marquee [RISQUE] impose sauvegarde + confirmation ecrite + fenetre d'intervention.

4. Installation et configuration P0

4.1 Recuperer les dependances (repertoire : racine du depot)

Objectif : venv Python + node_modules.

```
cd backend
python3 -m venv .venv && source .venv/bin/activate
pip install --upgrade pip
pip install -r requirements.txt
cd ../frontend
npm ci --no-audit --no-fund # ou : npm install --no-audit --no-fund
```

Attendu : import django OK (5.1.4), dossier frontend/node_modules present.

Diagnostic : relire l'erreur pip/npm ; verifier Python 3.12 et acces reseau ;

variante sandbox sans Postgres : voir arena/bootstrap.sh (requirements-sans-pg).

4.2 Créer le fichier d'environnement (SANS secret reel)

Objectif : backend/.env local a partir des exemples versionnés.

```
cp .env.example backend/.env.local.reference # modele prod (ne pas editer l'original)
cp backend/.env.example backend/.env # base commentee
# Editer backend/.env (placeholders, exemples NON sensibles) :
```

Contenu type DEMO (adapter, ne jamais commiter de vraie valeur) :

```
DJANGO_SECRET_KEY=<DJANGO_SECRET_KEY_DEMO> # generer : python3 -c "import secrets; print(secrets.token_urlsafe(50))"
DJANGO_DEBUG=False
DJANGO_ALLOWED_HOSTS=127.0.0.1, localhost
POSTGRES_HOST=127.0.0.1
POSTGRES_PORT=5432
POSTGRES_DB=injs_lmd_current
POSTGRES_USER=injs_user
POSTGRES_PASSWORD=<DB_PASSWORD_DEMO>
FRONTEND_URL=http://127.0.0.1:3000
PUBLIC_APP_URL=http://127.0.0.1:3000
CORS_ALLOWED_ORIGINS=
BADGE_BASE_URL=
PUBLIC_QR_SCAN_ENABLED=False
# E-mail dev : laisser vide (console). Prod : voir backend/.env.example (Mailcow).
```

Attendu : backend/.env present, ignore par Git (backend/.gitignore:210).

Diagnostics : `[code] git check-ignore -v backend/.env ; git status --short | head [/code]`

4.3 Base de données, services annexes, migrations, initialisation

Objectif : base joignable + 250 migrations appliquées, zero écriture demo.

```
pg_isready -h 127.0.0.1 -p 5432
psql -h 127.0.0.1 -p 5432 -U injs_user -d injs_lmd_current -c "SELECT current_database(), current_user;"
cd backend
python3 manage.py check
python3 manage.py showmigrations | grep -c "\\[X\\]" # attendu : 250
python3 manage.py showmigrations | grep "\\[\\]" | head # attendu : vide
```

Attendu : check 0 erreur (1 warning urls.W005 toléré), 250 x [X], 0 x [].

Si migrations en retard : `[code] python3 manage.py migrate --noinput [/code]` PUIS re-vérifier.

Diagnostics : host/port/user/db, mot de passe, droits ; ne JAMAIS flush/reset sans confirmation.

Bac à sable sans Postgres : USE_SQLITE=1 (voir scripts/start_dev.sh), SQLite = jetable uniquement.

4.4 Stockage, e-mail, tâches, cache

- Media/cache : `[code] mkdir -p backend/media backend/logs ; ls -ld backend/media backend/logs [/code]`
- E-mail dev : ne rien renseigner (console). Prod : EMAIL_HOST/PORT/USER/PASSWORD (Mailcow, port 587 STARTTLS recommandée).
- Cron (optionnel demo) : process_mobile_heartbeats, auto_close_pointages --skip-heartbeat, auto_sessions toutes les 5 min (service cron du Compose ou scripts/start_dev.sh cron).
- Cache : Redis si REDIS_URL défini, sinon fichier/mémoire (aucune action requise en demo).

4.5 Comptes, rôles et permissions (12 rôles figés, jamais d'extension sans métier valide)

Rôles : ADMIN, DIRECTION, CHEF_INJS_ADMIN, INJS_ADMIN, CHEF_SECRETARIAT, SECRETARIAT, FINANCE, ARCHIVE, ENCADRANT, SUPERVISEUR, FORMATEUR, AUDITEUR (= Etudiant).

Source : backend/authentication/models.py (User.Role). Groupes ROLE_* synchronisés.

Création (admin Django ou API) :

```
# Admin : http://127.0.0.1:8000/admin/ (superuser crée via entrypoint si DJANGO_SUPERUSER_* définis)
# API : POST /api/auth/users/ (Authorization: Bearer <token_admin>) {"username":..., "role":...}
# Mot de passe temporaire : <MOT_DE_PASSE_TEMPORAIRE_DEMO> à faire changer (must_change_password=True)
curl -s -X POST http://127.0.0.1:8000/api/auth/login/ -H "Content-Type: application/json" -d '{"username":"<US
```

Attendu : access+refresh reçus ; GET /api/auth/me/ 200 avec rôle correct.

Interdit : contourner l'authentification, partager un compte, stocker un secret durable.

4.6 Démarrage backend, frontend, Docker éventuel

Règle : API 8000, front 3000. Si port occupé : libérer, ne pas déplacer.

Option A (recommandee locale, lanceurs versionnes) :

```
bash arena/bootstrap.sh      # une fois apres chaque reinitialisation sandbox (idempotent)
bash arena/lancer-api.sh     # API 8000 (refuse si 8000 pris)
bash arena/lancer-front.sh   # front 3000 (strictPort, refuse si 3000 pris)
```

Option B (demo locale tout-en-un) :

```
./scripts/start_dev.sh      # frontend(3000, SPA via Django) + backend(8000) + cron
./scripts/start_dev.sh backend # API seule 8000
FRONTEND_MODE=dev ./scripts/start_dev.sh frontend # Vite HMR 3000 (+ API 8000 a cote)
```

Option C (Docker P3) : `[code] docker compose up --build [code] -> app http://localhost:8080, API http://localhost:8000, admin /admin/.`

Attendu : API 8000 repond, front 3000 affiche login, pas de bascule 5173.

5. Jeu de donnees de demonstration (fictif, prefixe DEMO)

Regle : aucune donnee reelle. E-mail en domaine reserve (exemple.*).

Toute donnee DEMO_* = supprimable de facon ciblée (voir section 8).

5.1 Etablissement + annee (ecran : Scolarite / admin ; tables : `scolarite_*`)

- Etablissement : INJS Marcory Abidjan (libelle existant, donnee demo).
- AnneeAcademique : libelle DEMO 2026-2027, code DEMO-2026, cloturee=False.
Injection : POST /api/scolarite/ref/anneeacademique/ (admin) ou admin Django.
Visible : selecteur d'annee Scolarite. Reset : archiver/cloturer, pas de DELETE si liee.
- Niveaux : L1/L2/L3 (ordre 1/2/3, cycle LICENCE) + M1/M2 si besoin.
- Parcours DEMO : DEMO-STAPS-MS (Management du Sport), ref_formation STAPS fictive.
- Semestres : S1..S6 rattaches aux niveaux ; S1/S2 utilises pour Awa KONE.
- Regimes/statuts : RegimeEtudes DEMO (Initial), StatutEtudiant DEMO (Regulier).
- RegleValidationLMD + RegleCalcul : seuils fictifs a saisir (ex. moyenne UE >= 10/20, compensation semestrielle active) et a faire valider (H3).

5.2 Maquette + UE/ECUE (ecran : Maquettes ; tables : `Maquette/UE/ECUE`)

- Maquette DEMO-MAQUETTE-STAPS-L1-2026 (annee 2026-2027, parcours DEMO-STAPS-MS, statut BROUILLON -> VALIDE -> ACTIVE via /maquettes/<pk>/{valider,activer}).
Seule une maquette VALIDEE/ACTIVE supporte un jury (contrainte verifiee en modele).
- Exemple S1 (30 ECTS fictifs) :
UE DEMO-UE101 Fondamentaux du sport (10 ECTS) : ECUE DEMO-EC101 Anatomie (5) + DEMO-EC102 Physiologie (5)
UE DEMO-UE102 Management de base (10 ECTS) : ECUE DEMO-EC103 Intro management (5) + DEMO-EC104 Droit du sport (5)
UE DEMO-UE103 Pratique + langue (10 ECTS) : ECUE DEMO-EC105 Pratique encadree (6) + DEMO-EC106 Anglais du sport (4)
- S2 : structure miroir DEMO-UE201..203 / DEMO-EC201..206 (meme logique, 30 ECTS).
- Injection : POST /api/scolarite/maquettes/creer/ puis /<pk>/ues/ puis /ues/<pk>/ecues/, ou ecrans Maquettes/MaquetteDetail. Visible : detail maquette + semestres/ECUE.
Reset : /<pk>/archiver/ + /cloner/ (pas de DELETE si jury lie).

5.3 Utilisateurs demo (ecran : Utilisateurs/CURP ; table : `authentication_user`)

- Creer via POST /api/auth/users/ (admin) avec mot de passe temporaire
<MOT_DE_PASSE_TEMPORAIRE_DEMO> + must_change_password=True. Ne jamais reutiliser les mots de passe de backend/seed_data.py.
- demo.admin (ADMIN), demo.scolarite (SECRETARIAT ou CHEF_SECRETARIAT),
demo.admission (INJS_ADMIN), demo.enseignant (FORMATEUR),
demo.jury (DIRECTION ou INJS_ADMIN, membre de jury), demo.finance (FINANCE),
awa.kone (AUDITEUR, creee apres admission).

- Visible : GET /api/auth/users/ + fiche compte. Reset : is_active=False (pas de DELETE).

5.4 Candidat fictif (tables : admissions_candidat/candidature/piece)

- Candidat : nom KONE, prenom Awa -- DONNEE DE DEMONSTRATION, sexe F, date_naissance 2002-05-14 (fictive), nationalite Ivoirienne (demo), e-mail awa.kone.demo@example.ci (domaine reserve, non reel), telephone +225 00 00 00 00 (fictif).
- Candidature DEMO-CAND-2026-001 : annee 2026-2027, RefFormation STAPS demo,

5.5 Suite du jeu demo (admission -> diplomation)

- Campagne : DEMO-CAMPAGNE-STAPS-2026 (OUVERTE, quotas fictifs admissibles/admis).
- Admission : decision ADMIS sur DEMO-CAND-2026-001 (reference DEMO-ADM-2026-001, date_limite_inscription future). Condition permet_inscription=true verifiee en modele.
- DossierEtudiant + matricule : INJS26-DEMO-0001 (attribue a la conversion).
- InscriptionAdministrative DEMO-INS-2026-001 (type PREMIERE, statut VALIDEE).
- Inscriptions pedagogiques : 6 ECUE S1 (puis S2) via /<pk>/pedagogie/generer/.
- Groupes : DEMO-TD1 (capacite 30) ; affectation historisee.
- Enseignants fictifs : DEMO-ENS-01/02/03 (FORMATEUR), specialites demo.
- Notes demo S1 : CC + Examen par ECUE, moyennes >= 10/20 (cas passant) ; prevoir aussi un cas compense et un cas rattrapage (autres etudiants DEMO_...).
- Jury : session DEMO-JURY-L1-S1-NORMALE (type NORMALE) + membres + decisions ADMIS.
- Diplome : DEMO-DIP-2026-0001 (DRAFT -> VALIDATED), PDF gele, registre annuel, token public DEMO-TOKEN-XXXX (verification sans donnee sensible).
- Finances demo : Tarification DEMO-TARIF-L1-2026 (montant fictif), echeancier 3 lignes, facture DEMO-FACT-0001, paiement DEMO-PAY-0001 confirme, quittance DEMO-QUIT-0001.
- Pour chaque donnee : methode = ecran cite section 1.3 OU endpoint cite OU admin Django ; dependances = ordre P1 (campagne avant candidature, etc.) ; attendu = statut visible dans l'ecran correspondant ; reset = transition inverse/target DELETE cible (section 8).

6. Procedure de demonstration metier P1 (check-list numerotee)

Format : Role | Action (ecran/API/commande) | Donnees demo | Resultat attendu | Preuve.

Si une etape etait indisponible, la mention exacte serait

« Non disponible dans le depot inspecte ». Ici toutes les etapes sont disponibles.

1. Ouverture campagne + referentiel. Role : demo.admin / demo.admission.

Action : Scolarite -> Maquettes (creer/valider/activer DEMO-MAQUETTE) ; Campagnes (creer DEMO-CAMPAGNE-STAPS-2026, transition OUVERTE).
API : POST /api/scolarite/maquettes/creer/, /<pk>/valider/, /<pk>/activer/ ;
POST /api/admissions/campagnes/, /<pk>/transition/ {"statut":"OUVERTE"}.
Attendu : maquette ACTIVE + campagne OUVERTE. Preuve : captures statuts.

2. Candidature d'Awa KONE. Role : demo.scolarite (ou auto-candidature si ouverte).

Action : Candidatures (creer candidat KONE Awa + candidature DEMO-CAND-2026-001, déposer 4 pieces factices, SOUMETTRE).
API : POST /api/admissions/candidats/, /candidatures/, /pieces/<id>/deposer/, /candidatures/<pk>/transition/ {"statut":"SOUMISE"}.
Attendu : statut SOUMISE, numero genere. Preuve : fiche candidature + pieces.

3. Verification + admission. Role : demo.admission.

Action : Controle des dossiers (verifier pieces -> PIECES_VALIDEES), passer EN_ETUDE -> ADMISSIBLE, creer Admission ADMIS (ref DEMO-ADM-2026-001).

API : /pieces/<id>/verifier/, /candidatures/<pk>/transition/,
POST /api/admissions/admissions/ + /<pk>/decision/ {"decision":"ADMIS"}.
Attendu : Admission ADMIS, permet_inscription=true. Preuve : fiche admission.

4. Activation etudiant + matricule. Role : demo.scolarite.

Action : Inscriptions -> inscrire_depuis_admission (ADMIS -> DossierEtudiant
INJS26-DEMO-0001 + compte awa.kone AUDITEUR).

API : POST /api/scolarite/inscriptions/depuis-admission/ {"admission":<id>}.

Attendu : DossierEtudiant + matricule affiches (FicheEtudiant). Preuve : fiche.

5. Inscription administrative + pedagogique. Role : demo.scolarite.

Action : valider inscription (BROUILLON -> VALIDEE), generer pedagogie S1
(6 ECUE), affecter groupe DEMO-TD1.

API : /inscriptions/<pk>/transition/ {"statut":"VALIDEE"},

/inscriptions/<pk>/pedagogie/generer/, /inscriptions/<pk>/affectations/.

Attendu : inscription VALIDEE + 6 lignes peda + groupe actif. Preuve : captures.

6. Groupes/enseignants. Role : demo.scolarite.

Action : Groupes (creer DEMO-TD1, capacite 30, repartition auto),
rattacher DEMO-ENS-01/02/03 aux ECUE (CM/TD/TP).

Attendu : effectifs coherents, 0 conflit EDT. Preuve : effectifs + EDT.

7. Saisie des resultats. Role : demo.enseignant.

Action : Evaluations -> sessions/composants -> saisie notes (CC + Examen
par ECUE, valeurs >= 10/20 pour Awa KONE ; cas compense/rattrapage sur
autres DEMO_*). API : /api/evaluations-academiques/composants/<pk>/notes/.

Attendu : notes historisees (EvaluationGradeHistory). Preuve : listes notes.

8. Calcul/validation, jury, credits. Role : demo.jury + demo.scolarite.

Action : resultats ECUE/UE/Semestre (lecture), creer SessionJury NORMALE
sur maquette verrouillee, ajouter membres, saisir decisions, transitions
PREPARATION->CONTROLE->CALCUL->DELIBERATION->DECISION->PV_GENERE->VALIDE->
VERROUILLE->PUBLIE, telecharger PV.

API : POST /api/jurys/sessions/, /<pk>/membres/, /<pk>/decisions/,
/<pk>/action/ {"action":"transition"}, /<pk>/pv/.

Attendu : session PUBLIEE + PV + 30 ECTS S1. Preuve : PV + relevés.

9. Eligibilite. Role : demo.scolarite.

Action : verifier inscription VALIDEE + decision jury session VERROUILLEE/
PUBLIEE + credits/UE + dettes (finances) avant diplome.

Attendu : 0 blocage pour Awa KONE (cas passant). Preuve : checklist signee.

10. Diplomation + documents. Role : demo.jury (creer/valider) + demo.scolarite.

Action : Graduation (diplome DEMO-DIP-2026-0001 creer -> valider),
telecharger PDF, controler registre annuel + verifier/<token>/ public.

API : POST /api/graduation/diplomes/creer/, /<pk>/valider/, /<pk>/pdf/,
GET /registres/, GET /verifier/<token>/.

Attendu : statut VALIDATED, PDF gele (SHA-256), registre renseigne,
verification publique OK sans donnee sensible. Preuve : PDF + registre.

11. Controle final. Role : demo.admin.

Action : FicheEtudiant (statut diplome), relevés (ReleveNotes versionnes),
credits totaux, audit (core_evenementaudit + journaux MaquetteJournal/
ReferentielJournal + JournalScolarite), droits par role (capabilities).

Attendu : coherence DB/API/UI, tracabilite complete. Preuve : exports + logs.

7. Validation, tests et criteres d'acceptation

7.1 Controles techniques (avant chaque demo)

```
cd backend
python3 manage.py check
python3 manage.py showmigrations | grep -c \"\\[X\\]\" # 250
curl -s http://127.0.0.1:8000/api/health/
curl -s -o /dev/null -w \"%{http_code}\\n\" http://localhost:3000/
tail -n 50 logs/injs_lmd.log
SECRET_KEY=ci-secret DEBUG=True python3 manage.py collectstatic --noinput
python3 manage.py test --noinput -v 2
cd ../frontend && npm run test:run
```

Attendu : check 0 erreur, health ok, front 200, suites vertes.

Reference CI : PostgreSQL 16 / Python 3.12 ; WhiteNoise : collectstatic avant tests templates.

7.2 Controles fonctionnels (pendant la demo)

Statuts : campagne OUVERTE, candidature SOUMISE->PIECES_VALIDEES->ADMIS,
admission ADMIS, inscription VALIDEE, notes historisees, jury PUBLIE,
diplome VALIDATED, registre renseigne.

Coherence : somme ECTS S1 = 30 ; moyennes ECUE->UE->Semestre recalculees ;
droits par role (FINANCE isole, ADMIN global) ; documents generes (PV, releve, diplome).

7.3 Cas negatifs essentiels (a jouer avec des DEMO_* secondaires)

- N1 Candidature incomplete : piece manquante -> transition refusee (PIECES_INCOMPLETES).**
- N2 Admission refusee : decision REFUSEE -> permet_inscription=false, conversion bloquee.**
- N3 Paiement absent : facture impayee -> relance visible, eligibilite diplome reservee (H4).**
- N4 Note invalide : note hors 0-20 -> 400 + historique inchangé.**
- N5 Credits insuffisants : UE < seuil -> decision AJOURNE, diplome refuse (clean() lève).**
- N6 Diplomation refusee : sans inscription VALIDEE ou sans jury VERROUILLE/PUBLIE -> VALIDATED bloque.**
- N7 Campagne cloturee : nouvelle candidature refusee.**
- N8 Annee cloturee : nouvelle inscription/diplome refuse(e).**
- N9 Double inscription VALIDEE meme annee : contrainte unicite violee.**
- N10 Maquette non verrouillee : session jury refusee.**
- N11 Acces interdit : AUDITEUR sur /api/auth/users/ -> 403 ; FINANCE hors finance -> 403.**
- N12 Token diplome inconnu : /verifier/INCONNU/ -> 404 sans fuite de donnees.**

Reference : backend/tests_e2e_injs_2026_2027.py annonce 12 cas negatifs + RBAC.

7.4 Tableau Test / Precondition / Action / Resultat attendu / Preuve

- T1 Health | services demarres | GET /api/health/ | status ok + database ok | JSON capture.**
- T2 Login roles | comptes demo | POST /api/auth/login/ x6 roles | 6 JWT + /me/ OK | tokens (non stockes).**
- T3 Campagne | annee+formation | transition OUVERTE | statut OUVERTE | capture.**
- T4 Candidature | campagne OUVERTE | creer+soumettre+pieces | SOUMISE | fiche.**
- T5 Admission | pieces validees | decision ADMIS | permet_inscription | fiche.**
- T6 Inscription | admission ADMIS | depuis-admission + VALIDEE | matricule | FicheEtudiant.**
- T7 Pedagogie | inscription VALIDEE | generer S1 | 6 lignes | liste peda.**
- T8 Paiement | facture | confirmer paiement | quittance | quittance PDF.**
- T9 Notes | composants | saisie CC/Exam | moyennes | listes notes.**
- T10 Jury | notes + maquette verrouillee | decisions + PUBLIE | PV | PV PDF.**
- T11 Diplome | jury PUBLIE + insc VALIDEE | creer+valider+pdf | VALIDATED + registre | PDF + registre.**
- T12 Verification | diplome VALIDATED | GET /verifier/<token>/ | fiche publique | capture.**

8. Exploitation, reinitialisation et depannage

8.1 Reset sur de la demo (JAMAIS flush/drop/truncate)

Principe : sauvegarde d'abord, suppression ciblée DEMO_* ensuite, reimport, verification.

```
bash scripts/backup_injs_lmd.sh
ls -lh ~/Backups/INJS-LMD/database/ | tail -n 5
# Verification avant reparation (obligatoire) :
psql -h 127.0.0.1 -p 5432 -U injs_user -d injs_lmd_current -c "SELECT count(*) FROM admissions_candidature WHERE nu
# Suppression ciblée (ordre inverse des dependances : diplomes -> jury -> notes -> peda -> inscription -> admission
# Puis rejouer la section 6. Verifier : health + statuts + registre.
```

Commandes INTERDITES sans confirmation explicite : DROP DATABASE/TABLE, TRUNCATE, DELETE massif, manage.py flush, docker compose down -v, suppression de volumes/migrations.

8.2 Sauvegarde/restauration (script officiel : scripts/backup_injs_lmd.sh)

Sauvegarde : base injs_lmd_current -> ~/Backups/INJS-LMD/database/<base>_<date>.dump + .sha256.

Verification : dump non vide + [code] pg_restore --list <fichier.dump> [/code] + SHA-256.

Sauvegarde historique : injs_lmd_avant_reinitialisation_2026-08-29.dump (hors depot Git, ne jamais supprimer sans autorisation).

Restauration [RISQUE] : uniquement sur base dediee + confirmation écrite + fenetre convenue ; documenter source, cible, SHA-256 avant/apres. Git = restauration du code (jamais reset --hard / clean -fd / push --force sans autorisation ; préserver les modifications non commitees).

8.3 Les 10 problemes probables (verification AVANT reparation)

1. Deps Python : verifier venv + requirements ; reparer : recreer venv, reinstaller.
2. Port occupe (3000/8000) : verifier ss/lsof ; reparer : pkill cible (manage.py runserver / vite), jamais --port.
3. .env absent/mauvais : verifier git check-ignore + contenu placeholders ; reparer : recopier exemples.
4. Postgres injoignable : verifier pg_isready + host 5432 (pas 5436) ; reparer : demarrer service, corriger HOST/PORT/USER.
5. Migrations en retard : verifier showmigrations [] ; reparer : migrate --noinput puis re-verifier.
6. Permissions/403 : verifier /me/ + capabilities + role ; reparer : attribuer role/capacite, pas de contournement.
7. Media/statiques 404 : verifier backend/media + collectstatic + WhiteNoise ; reparer : mkdir/collectstatic.
8. E-mail bloquant : verifier EMAIL_* vides en dev ; reparer : console dev, SMTP prod seulement.
9. PDF/Excel en echec : verifier reportlab/openpyxl/python-docx ; reparer : pip install -r requirements.txt.
10. Auth JWT/MFA : verifier login + refresh + throttles (THROTTLE_LOGIN_RATE=5/min) ; reparer : attendre/fenetre, verifier horloge, jamais desactiver les controles.

9. Ecart, decisions et backlog

9.1 Ecart constatés (objectif candidature->diplomation vs capacites)

E1 seed_data.py historique inutilisable en l'etat (mots de passe en clair, referentiel SYGEP-CPFAE).

Priorite P0. Impact : fausse demo / faille si execute. Resolution : ne pas executer ; jeu DEMO_* section 5 + creation via API/admin. Fini quand : seed historique isole/documente.

E2 Seuil/metier LMD non chiffres dans le depot (compensation, rattrapage, seuils, dettes).

P1. Impact : decisions jury/diplome a interpreter. Resolution : saisir RegleValidationLMD/ RegleCalcul demo + validation INJS. Fini quand : note de cadrage signee.

E3 Notifications dispersees (pas de module unifie). P1. Impact : notification d'admission manuelle.

Resolution : statut + capture ; backlog module notifications. Fini quand : gabarits valides.

E4 Comptabilite generale absente (app a creer). P2. Impact : aucun sur parcours etudiant.

Resolution : rapprochements L6 suffisent en demo. Fini quand : perimetre comptable cadre.

E5 Doublons historiques (/api/juries/ + /api/jurys/, sigles IsDFRC, base qr_badge, images qr-badge-*).

P3. Impact : confusion d'integration. Resolution : utiliser /api/jurys/ ; bascule par lots
reversibles derriere flags (DA-12/ADR-005). Fini quand : ADR de bascule validee.

E6 Warning urls.W005 (namespace dashboard_api non unique). P2. Impact : reverse() ambigu.

Resolution : dedupliquer le namespace. Fini quand : check 0 warning.

E7 Compose local vs CI (Postgres 15 image historique vs 16 CI ; README §5 dit 15/16).

P3. Impact : aucun en local (16.14 constate). Resolution : aligner les docs/images. Fini OK.
Aucune modification applicative n'a ete effectuee pour ce manuel (livrable = fichier seul) ;
si une correction devient indispensable a la demo, la documenter ici avec fichier, justification
et retour arriere (git diff + restore), puis demander confirmation avant execution.

9.2 Decisions fonctionnelles a faire valider par l'INJS

D1 Annee officielle + calendrier + sessions (normale/rattrapage/rectification).

D2 Offre réelle (UFR/filiere/mention/parcours/semestres/UE/ECUE/ECTS) vs parcours DEMO fictif.

D3 Regles de passage/validation/compensation/rattrapage + seuils chiffres.

D4 Conditions de diplomation (credits, UE, statut administratif, dettes, jury).

D5 Modeles graphiques (attestation, releve, diplome, PV) + mentions/signataires.

D6 Tarifs/frais/echeances/exonerations + quittances/remboursements.

D7 Droits de jury (composition, quorum, publication, rectification) + archivage PV.

D8 Conservation/audit (durees, registre, acces) + protection des donnees.

FIN DU MANUEL -- Document autonome. En cas d'incertitude : STOP, presenter le risque,
demander confirmation. Mieux vaut demander que detruire une donnee.

(Les sections 0 a 9 sont toutes presentes ; plan P0/P1/P2/P3 ; ordre
candidature -> admission -> inscription -> evaluations -> jury -> diplomation ;
donnees fictives DEMO_* ; aucun secret reel.)